

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.N o	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

CSA0509 – DATABASE
MANAGEMENT SYSTEM

CO3-AT2

CASE STUDY

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Dr. R Kesavan

Submitted on:

12-08-2026

Q1. Online Shopping Platform

Question:

Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Answer:

Step 1: Functional Dependencies

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$
- $(\text{OrderID, ProdID}) \rightarrow \text{Qty}$

Therefore, the candidate key is: (OrderID, ProdID)

Step 2: 1NF

All attributes contain atomic values, so the relation is already in **1NF**.

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Primary Key: (OrderID, ProdID)

Step 3: 2NF

There are partial dependencies:

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$

These depend only on part of the composite key.

Decompose into:

CUSTOMER

Customer(CustID, CustName)

Primary Key: CustID

PRODUCT

Product(ProdID, ProdName, Price)

Primary Key: ProdID

ORDER

Order(OrderID, CustID)

Primary Key: OrderID

ORDER_DETAIL

OrderDetail(OrderID, ProdID, Qty)

Primary Key: (OrderID, ProdID)

Step 4: 3NF

There is a transitive dependency:

$\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$

Since CustName is already separated into Customer, the transitive dependency is removed.

Final 3NF Schema

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

ORDER(OrderID, CustID)

ORDER_DETAIL(OrderID, ProdID, Qty)

Conclusion: The above decomposition is in **3NF**, reduces redundancy, and prevents insertion, update, and deletion anomalies.

Q2. Hospital Patient Management

Question:

A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Answer:

Functional Dependencies

- $\text{PID} \rightarrow \text{PName}, \text{DocID}, \text{WardName}$
- $\text{DocID} \rightarrow \text{DocName}, \text{Specialization}$

Candidate key: PID

Problems

1. Update Anomaly

If a doctor's specialization changes, it must be updated for every patient treated by that doctor.

2. Insertion Anomaly

A new doctor cannot be stored unless at least one patient is assigned to that doctor.

3. Deletion Anomaly

If the last patient of a doctor is deleted, the doctor's information may also be lost.

1NF

The relation contains atomic values:

Patient(PID, PName, DocID, DocName, Specialization, WardName)

2NF

The key is a single attribute PID, so there is **no partial dependency**.

Therefore, it is already in 2NF.

3NF

There is a transitive dependency: $PID \rightarrow DocID \rightarrow DocName, Specialization$

Decompose:

PATIENT

Patient(PID, PName, DocID, WardName)

DOCTOR

Doctor(DocID, DocName, Specialization)

BCNF Verification

For Patient: $PID \rightarrow PName, DocID, WardName$

PID is a candidate key.

For Doctor: $DocID \rightarrow DocName, Specialization$

DocID is a candidate key.

Therefore, every determinant is a candidate key.

Final BCNF

PATIENT(PID, PName, DocID, WardName)

DOCTOR(DocID, DocName, Specialization)

Hence, the decomposed relations satisfy BCNF.

Q3. University Enrollment

Question:

A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF.

Answer:

Functional Dependencies

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

Candidate key: (SID, CourseID)

1NF : All attributes are atomic.

Therefore: Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade) is in **1NF**.

Partial Dependencies

We have: $SID \rightarrow SName$ and

$CourseID \rightarrow CourseName, Faculty$

These depend on only part of the composite key (SID, CourseID).

Therefore, the relation violates **2NF**.

Decomposition into 2NF

STUDENT

Student(SID, SName)

Primary Key: SID

COURSE

Course(CourseID, CourseName, Faculty)

Primary Key: CourseID

ENROLLMENT

Enrollment(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

Final Answer

STUDENT(SID, SName)

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT(SID, CourseID, Grade)

Thus, all non-key attributes are fully dependent on the complete key of their respective relation, satisfying **2NF**.

Q4. Airline Booking

Question:

Booking(FlightNo, PAssID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Answer:

Functional Dependencies

Assuming a flight number identifies a scheduled flight:

- FlightNo $\rightarrow$ DeptCity, ArrCity
- (FlightNo, Date) $\rightarrow$ Flight details
- PAssID $\rightarrow$ PassName
- (FlightNo, Date, PAssID) $\rightarrow$ Seat

Candidate key: (FlightNo, Date, PAssID)

1NF

All attributes are atomic.

2NF

Partial dependencies: $\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

And $\text{PassID} \rightarrow \text{PassName}$

These depend on only parts of the composite key.

Decompose:

FLIGHT

$\text{Flight}(\text{FlightNo}, \text{DeptCity}, \text{ArrCity})$

PASSENGER

$\text{Passenger}(\text{PassID}, \text{PassName})$

BOOKING

$\text{Booking}(\text{FlightNo}, \text{Date}, \text{PassID}, \text{Seat})$

Primary Key: $(\text{FlightNo}, \text{Date}, \text{PassID})$

BCNF

For Flight:

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

FlightNo is a candidate key.

For Passenger:

$\text{PassID} \rightarrow \text{PassName}$

PassID is a candidate key.

For Booking: $(\text{FlightNo}, \text{Date}, \text{PassID}) \rightarrow \text{Seat}$

The determinant is the candidate key.

Final BCNF Schema

$\text{FLIGHT}(\text{FlightNo}, \text{DeptCity}, \text{ArrCity})$

$\text{PASSENGER}(\text{PassID}, \text{PassName})$

$\text{BOOKING}(\text{FlightNo}, \text{Date}, \text{PassID}, \text{Seat})$

Hence, the schema is normalized to **BCNF**.

Q5. HR System for a Multinational Company

Question:

Analyze an HR system schema and identify all functional and transitive dependencies and normalize to 3NF.

Answer:

A suitable HR relation is:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, CountryID, CountryName)

Functional Dependencies

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}, \text{CountryID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

$\text{ManagerID} \rightarrow \text{ManagerName}$

$\text{CountryID} \rightarrow \text{CountryName}$

Transitive Dependencies

We have: $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

$\text{EmpID} \rightarrow \text{CountryID} \rightarrow \text{CountryName}$

These are transitive dependencies.

1NF: All values are atomic.

2NF: The candidate key is EmpID, which is a single attribute.

Therefore, there is no partial dependency and the relation is in 2NF.

3NF Decomposition

EMPLOYEE

Employee(EmpID, EmpName, DeptID, ManagerID, CountryID)

DEPARTMENT

Department(DeptID, DeptName, ManagerID)

MANAGER

Manager(ManagerID, ManagerName)

COUNTRY

Country(CountryID, CountryName)

Final 3NF

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, CountryID)

DEPARTMENT(DeptID, DeptName, ManagerID)

MANAGER(ManagerID, ManagerName)

COUNTRY(CountryID, CountryName)

This removes transitive dependencies and reduces redundancy.

Q6. Library Management

Question:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and convert to 4NF.

Answer:

Functional Dependencies

Assume:

MemberID $\rightarrow$ MemberName

BookID $\rightarrow$ Title, Author, Genre

(MemberID, BookID) $\rightarrow$ BorrowDate

Candidate key:

(MemberID, BookID)

However, suppose a member can independently have multiple **books** and multiple **authors/genres** associated with the member.

This creates independent multivalued dependencies.

Multivalued Dependencies

MemberID $\twoheadrightarrow$ BookID

MemberID $\twoheadrightarrow$ Author

The independent multivalued facts cause redundancy.

4NF Rule

A relation is in 4NF if every non-trivial multivalued dependency has a superkey as its determinant.

Decomposition

MEMBER

Member(MemberID, MemberName)

BOOK

Book(BookID, Title, Author, Genre)

BORROW

Borrow(MemberID, BookID, BorrowDate)

Final 4NF Schema

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Author, Genre)

BORROW(MemberID, BookID, BorrowDate)

The independent multivalued dependencies are separated, so the resulting relations satisfy **4NF**.

Q7. Telecom Billing Schema

Question:

Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Answer:

Consider:

Billing(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, Rate, CallID, CallDuration, Amount)

Functional Dependencies

BillID $\rightarrow$ CustomerID, PlanID

CustomerID $\rightarrow$ CustomerName, PhoneNo

PlanID $\rightarrow$ PlanName, Rate

CallID $\rightarrow$ CallDuration

(BillID, CallID) $\rightarrow$ Amount

Candidate Key: Because a bill can contain multiple calls:

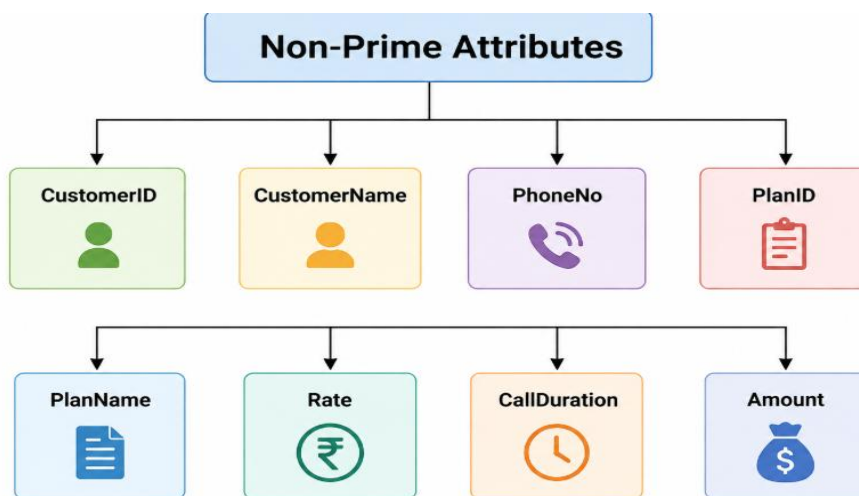
(BillID, CallID) is the candidate key.

Primary Key

PRIMARY KEY = (BillID, CallID)

Prime Attributes

The prime attributes are: BillID, CallID



Partial Dependencies

BillID $\rightarrow$ CustomerID, PlanID

CallID $\rightarrow$ CallDuration

These depend on only part of the composite key.

Transitive Dependencies

BillID $\rightarrow$ CustomerID $\rightarrow$ CustomerName, PhoneNo

and

BillID $\rightarrow$ PlanID $\rightarrow$ PlanName, Rate

Conclusion

The original relation contains both **partial and transitive dependencies**, so it should be decomposed into smaller relations before normalization.

Q8. Retail Inventory

Question:

A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Qty, Warehouse). Apply complete normalization.

Answer:

Functional Dependencies

$PID \rightarrow PName, SupplierID, Category, Price$

$SupplierID \rightarrow SupplierName$

$(PID, Warehouse) \rightarrow Qty$

Candidate key: (PID, Warehouse)

1NF: All attributes contain atomic values.

Therefore: Product(PID, PName, SupplierID, SupplierName, Category, Price, Qty, Warehouse) is in 1NF.

2NF

There are partial dependencies:

$PID \rightarrow PName, SupplierID, Category, Price$

$SupplierID \rightarrow SupplierName$

PID is only part of the composite key (PID, Warehouse).

So decompose.

PRODUCT

Product(PID, PName, SupplierID, Category, Price)

Primary Key: PID

SUPPLIER

Supplier(SupplierID, SupplierName)

Primary Key: SupplierID

INVENTORY

Inventory(PID, Warehouse, Qty)

Primary Key: (PID, Warehouse)

3NF

There is a transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

This is removed by creating the Supplier table.

Final 3NF Schema

PRODUCT(PID, PName, SupplierID, Category, Price)

SUPPLIER(SupplierID, SupplierName)

INVENTORY(PID, Warehouse, Qty)

Keys

PRODUCT:

PK = PID

FK = SupplierID

SUPPLIER:

PK = SupplierID

INVENTORY:

PK = (PID, Warehouse)

FK = PID

Thus, the inventory system is normalized up to **3NF**.

Q9. School ERP – BCNF

Question:

Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Answer:

A suitable poorly normalized schema is:

School(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Functional Dependencies

$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$.

$\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$

$\text{TeacherID} \rightarrow \text{TeacherName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{Grade}$

Candidate key: $(\text{StudentID}, \text{SubjectID})$

Problems

There are partial dependencies:

$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

There are transitive dependencies:

$\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$

$\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{TeacherID} \rightarrow \text{TeacherName}$

Decomposition

STUDENT

Student(StudentID, StudentName, ClassID)

PK: StudentID

CLASS

Class(ClassID, ClassName, TeacherID)

PK: ClassID

TEACHER

Teacher(TeacherID, TeacherName)

PK: TeacherID

SUBJECT

Subject(SubjectID, SubjectName)

PK: SubjectID

ENROLLMENT

Enrollment(StudentID, SubjectID, Grade)

PK: (StudentID, SubjectID)

BCNF Verification

For every relation, the determinant is a candidate key:

StudentID $\rightarrow$ StudentName, ClassID

StudentID is a candidate key.

ClassID $\rightarrow$ ClassName, TeacherID

ClassID is a candidate key.

TeacherID $\rightarrow$ TeacherName

TeacherID is a candidate key.

SubjectID $\rightarrow$ SubjectName

SubjectID is a candidate key.

(StudentID, SubjectID) $\rightarrow$ Grade

The composite determinant is the candidate key.

Final BCNF Design

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

SUBJECT(SubjectID, SubjectName)

ENROLLMENT(StudentID, SubjectID, Grade)

Justification: Every non-trivial functional dependency has a determinant that is a candidate key.

Therefore, the decomposed schema satisfies **BCNF**.

Q10. Movie Streaming Platform – 5NF

Question:

Given a movie streaming platform schema, identify join dependencies and decompose it into 5NF with lossless-join verification.

Answer:

Consider: Streaming(UserID, MovieID, PlatformID)

Suppose a user can watch a movie, a movie can be available on a platform, and a user can use a platform independently.

Possible Join Dependencies

The relation contains three independent relationships:

User $\leftrightarrow$ Movie

Movie $\leftrightarrow$ Platform

User $\leftrightarrow$ Platform

Therefore, the relation can be decomposed into:

USER_MOVIE

UserMovie(UserID, MovieID)

MOVIE_PLATFORM

MoviePlatform(MovieID, PlatformID)

USER_PLATFORM

UserPlatform(UserID, PlatformID)

5NF Decomposition

USER_MOVIE(UserID, MovieID)

MOVIE_PLATFORM(MovieID, PlatformID)

USER_PLATFORM(UserID, PlatformID)

Join Dependency

The original relation can be reconstructed using:

Streaming = UserMovie $\bowtie$ MoviePlatform $\bowtie$ UserPlatform

The join dependency is:

$\ast(\text{UserID}, \text{MovieID}),$
 $(\text{MovieID}, \text{PlatformID}),$
 $(\text{UserID}, \text{PlatformID})$

Lossless Join Verification

Perform the natural join: **UserMovie** $\bowtie$ **MoviePlatform** $\bowtie$ **UserPlatform**

If the original valid tuples are obtained without generating spurious tuples, the decomposition is lossless.

Final Answer

The relation is decomposed into:

USER_MOVIE(UserID, MovieID)

MOVIE_PLATFORM(MovieID, PlatformID)

USER_PLATFORM(UserID, PlatformID)

This removes non-trivial join dependencies and gives a **5NF design**.

Code of Conduct:

I _ M o n i k a . R (1 9 2 3 2 4 2 8 1) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Monika.R

RUBRICS FOR CALCULATION:

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand